



Khulna University of Engineering & Technology

Department of Computer Science and Engineering

Project Report

BanglaCare: An Entity-Aware NLP System for Triage of Bengali Consumer Health Queries

Course Code: CSE 4122

Course Title: Natural Language Processing Laboratory

Submitted by

MD Tahmid Islam Roll: 2107107

Adit Mugdha Das Roll: 2107118

Submitted to

Dr. K. M. Azharul Hasan

Professor

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Mr. Md. Nazirulhasan Shawon

Assistant Professor

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Date of Submission: September 19, 2026

Contents

1	Introduction	2
2	Background and Related Work	2
3	Dataset, Corpus and Preprocessing	3
3.1	Datasets	3
3.2	Medical Corpus Construction	4
3.3	Preprocessing and Leakage Prevention	5
4	Proposed Methodology	5
4.1	Overall Architecture	5
4.2	FastText–BiLSTM Encoder	6
4.3	BanglaBERT Encoder	7
4.4	NER and Entity-Aware Severity Prediction	8
5	Training and Experimental Setup	10
5.1	Training Procedure	10
5.2	Experimental Configuration and Calibration	12
6	Results and Evaluation	13
6.1	NER and Severity Performance	13
6.2	Severity Errors, Calibration and Human Review	14
7	Error Analysis and Discussion	15
8	Limitations and Future Work	16
9	Conclusion	16

1 Introduction

Consumer health queries written in Bengali are often difficult to process automatically because users may describe the same medical problem using formal Bangla, informal expressions, English medical terms, Banglish transliteration, or spelling variations. A single query may also contain several types of medical information, such as symptoms, medicines, dosage, age, existing conditions, specialists, or medical procedures. Therefore, a useful health query processing system should not only classify the complete sentence, but also identify the medically important information contained in it.

BanglaCare addresses two related NLP tasks. The first is *medical named entity recognition*, where medically relevant spans are extracted from the query. The second is *severity triage*, where the complete query is classified into one of four classes: **Emergency**, **Urgent**, **Routine**, or **General Query**. Instead of treating these tasks as completely independent, the proposed system uses information from the NER branch to support severity classification.

The project investigates two different contextual encoders. The first is a lightweight FastText–BiLSTM architecture that combines general Bengali word knowledge with domain-specific medical embeddings. The second uses pretrained BanglaBERT to obtain contextual Transformer representations. Both models use the same general downstream design consisting of an NER head with a CRF layer and an entity-aware severity classification head.

The main objectives of the project are:

- to identify seven medical entity categories from Bengali consumer health queries;
- to classify queries into four severity levels;
- to incorporate NER-derived entity probabilities into severity prediction using entity-aware attention;
- to compare FastText–BiLSTM and BanglaBERT as alternative contextual encoders;
- to evaluate both models using held-out NER and severity test sets;
- to provide calibrated confidence scores and flag uncertain cases for possible human review.

The main contribution of BanglaCare is the integration of medical entity information with severity triage in a shared multi-task architecture. The project also includes a domain-specific Medical FastText model, duplicate-aware data splitting, held-out-data protection during corpus construction, a comparison between lightweight and Transformer encoders, and confidence-based review routing.

2 Background and Related Work

Named Entity Recognition (NER) identifies important text spans and assigns them to predefined categories. BanglaCare recognizes seven medical entity types: Symptom, Medicine, Dosage, Age, Health Condition, Specialist, and Medical Procedure. Entity boundaries are represented using BIO tagging, where B marks the beginning of an entity, I marks a continuation, and O represents a non-entity token. This gives a total of

$$7 \times 2 + 1 = 15 \tag{1}$$

BIO labels.

The lightweight model uses FastText embeddings followed by a Bidirectional LSTM. FastText represents words using both whole-word and character-level subword information, which is useful for Bengali medical terms and spelling variations [1]. The BiLSTM then processes the sequence in both directions to produce context-aware token representations.

For NER decoding, BanglaCare uses a Conditional Random Field (CRF). The model first produces emission scores for the BIO labels, while the CRF also considers learned transition scores between neighbouring tags. The sequence score can be written as

$$S(y) = \sum_{i=1}^n E_{i,y_i} + \sum_{i=1}^n T_{y_{i-1},y_i}, \quad (2)$$

where E denotes emission scores and T denotes transition scores. Viterbi decoding selects the highest-scoring BIO sequence.

The second encoder is BanglaBERT, a pretrained Transformer model. Transformers use self-attention to model relationships between tokens [2]. BanglaBERT produces contextual subword representations, which are aligned back to the original words before being passed to the NER and severity heads.

For severity prediction, BanglaCare uses entity-aware attention. The classifier uses both contextual token representations and soft probabilities indicating the likelihood of each medical entity type, allowing medically relevant tokens to contribute more strongly to the final severity decision.

Previous NER systems have commonly used BiLSTM-CRF architectures, while pretrained Transformers are now widely used for both token-level and sentence-level NLP tasks. BanglaCare combines these ideas in a multi-task setting where medical entity information is also used to support severity classification.

3 Dataset, Corpus and Preprocessing

3.1 Datasets

Four data sources are used in BanglaCare. Bangla-HealthNER and the Bangla Healthcare Severity Dataset provide supervised labels for the two main tasks, while BanglaHealth and BanglaCHQ-Summ provide additional domain-specific text for constructing the medical corpus.

Table 1: Datasets used in BanglaCare.

Dataset	Content	Size / Split	Role
Bangla-HealthNER	Bangla health sentences annotated with medical entities.	25,426 train, 3,178 validation, 3,179 test	NER supervision
Bangla Healthcare Severity Dataset	Consumer health queries labelled according to four severity classes.	5,263 total: 4,211 train, 526 validation, 526 test	Severity supervision
BanglaHealth	Bangla medical and health-related text containing diseases, symptoms, treatments, medicines, and general health information.	External text source	Medical corpus
BanglaCHQ-Summ	Consumer-health questions and related health text representing natural health-query language.	External text source	Medical corpus

The two supervised tasks use the target labels shown in Table 2.

Table 2: Target labels used in BanglaCare.

Task	Labels
Medical NER	Symptom, Medicine, Dosage, Age, Health Condition, Specialist, Medical Procedure
Severity Triage	Emergency, Urgent, Routine, General Query

Bangla-HealthNER provides token-level supervision for the medical NER task. The seven entity categories are converted into BIO tags. Each entity type has a beginning tag (B-) and an inside tag (I-), together with the outside tag O. Therefore, the NER output space contains

$$7 \times 2 + 1 = 15 \quad (3)$$

BIO labels.

The severity dataset provides query-level supervision for four-class medical triage. Its class distribution is 1,353 Emergency, 1,461 Urgent, 1,268 Routine, and 1,181 General Query samples. The final split contains 4,211 training queries, 526 validation queries, and 526 test queries.

The severity dataset also contains an **Action Needed** field. This field is excluded from the model input because it can directly indicate the intended severity class and would therefore introduce target leakage.

BanglaHealth and BanglaCHQ-Summ are not used as labelled NER or severity datasets. Instead, they provide additional text for domain-specific FastText training. BanglaHealth contributes general medical vocabulary and health information, while BanglaCHQ-Summ contributes consumer-health language closer to the way users naturally formulate health-related questions.

3.2 Medical Corpus Construction

The Medical FastText model is trained on a separate domain-specific corpus. Using only the HealthNER and severity training data would provide relatively limited vocabulary coverage, so additional medical text from BanglaHealth and BanglaCHQ-Summ is included.

The medical corpus is constructed from:

- BanglaHealth,
- BanglaCHQ-Summ,
- the training portion of Bangla-HealthNER, and
- the training portion of the severity dataset.

The text is cleaned and deduplicated before FastText training. Held-out validation and test examples are not intentionally added to the corpus. The final corpus contains approximately 298,447 lines and 131,406 unique tokens.

Figure 1 summarizes how the datasets are used in the project.

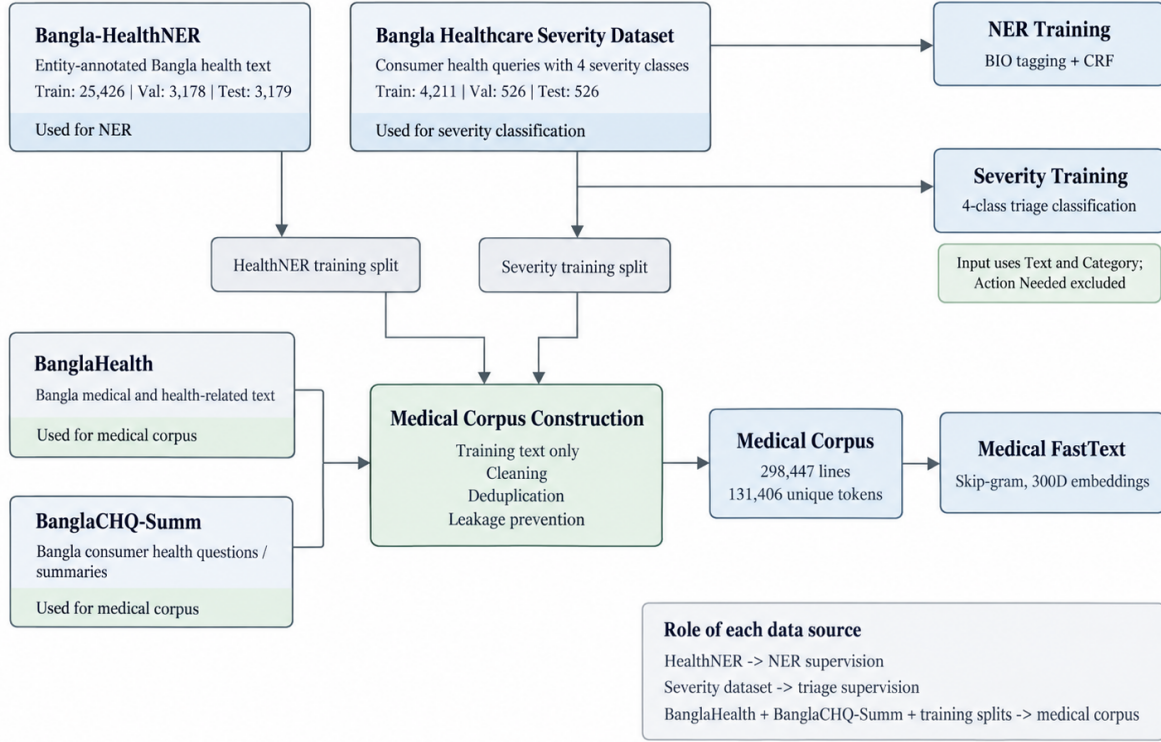


Figure 1: Dataset usage and medical corpus construction.

3.3 Preprocessing and Leakage Prevention

The preprocessing pipeline is designed to standardize noisy user text without removing medically important information. The main operations include Unicode normalization, whitespace normalization, text cleaning, and tokenization. Tokens representing medicine names, dosages, ages, procedures, English medical terms, and mixed Bangla–English expressions are preserved whenever possible.

Duplicate and near-duplicate handling is important for the severity dataset. Very similar queries are prevented from being independently placed in both training and test data, reducing the chance that the model obtains an artificial advantage through memorization. Target-derived information such as **Action Needed** is also removed from the model input.

The Transformer variant requires one additional alignment step. HealthNER annotations are defined at the original word level, while BanglaBERT uses subword tokenization. Therefore, normalization is applied while preserving original word boundaries, and the contextual representation of the first BanglaBERT subword corresponding to each original word is selected for downstream NER and severity processing. This preserves the alignment between each original word and its BIO annotation.

4 Proposed Methodology

4.1 Overall Architecture

BanglaCare is designed as a multi-task NLP system with two related outputs: medical Named Entity Recognition (NER) and query-level severity classification. After preprocessing, the input query is passed through a contextual encoder. Two encoder alternatives are investigated: a lightweight

FastText–BiLSTM encoder and a pretrained BanglaBERT encoder.

The encoder produces one contextual representation for each input token. These representations are shared by two downstream branches. The first branch performs medical NER using a linear BIO classifier followed by a Conditional Random Field (CRF). The second branch performs severity classification. Instead of using only the contextual representation, the severity branch also receives soft medical entity probabilities generated by the NER branch. These probabilities are used inside an entity-aware attention mechanism.

The complete architecture is shown in Figure 2.

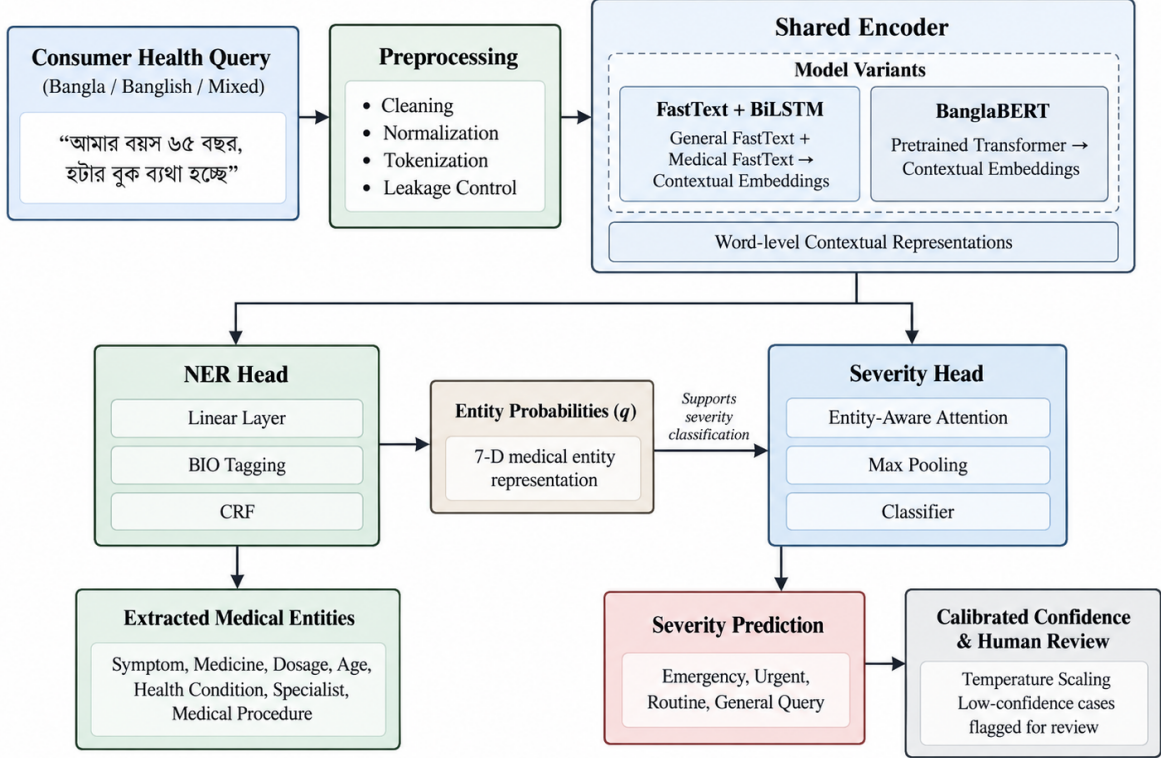


Figure 2: Overall architecture of the BanglaCare system.

4.2 FastText–BiLSTM Encoder

The lightweight architecture begins with two FastText representations for each input token. A pretrained General FastText model provides general Bengali language information, while a Medical FastText model trained on the constructed medical corpus provides domain-specific health information. Each model produces a 300-dimensional embedding. The two vectors are concatenated:

$$\mathbf{x}_i = [\mathbf{x}_i^{general}; \mathbf{x}_i^{medical}] \in \mathbb{R}^{600}. \quad (4)$$

Thus, every token initially has a 600-dimensional representation containing both general and medical language information.

The concatenated vector is then passed through a learnable projection:

$$600 \rightarrow 256.$$

The projection is followed by Layer Normalization, GELU activation, and dropout. Layer Normalization stabilizes the representation, GELU introduces non-linearity, and dropout reduces overfitting. The resulting sequence is processed by a two-layer Bidirectional LSTM. The forward LSTM reads the sequence from left to right, while the backward LSTM reads it from right to left. Each direction uses 192 hidden units, giving

$$192 + 192 = 384$$

dimensions for each token.

Therefore, the FastText vectors provide the initial word representation, whereas the BiLSTM converts them into contextual representations in which a token is influenced by the surrounding words.

The complete encoder can be summarized as

$$\text{General FastText}_{300} + \text{Medical FastText}_{300} \rightarrow 600 \rightarrow 256 \rightarrow \text{BiLSTM} \rightarrow 384.$$

4.3 BanglaBERT Encoder

The second architecture replaces the complete FastText–projection–BiLSTM pipeline with the pre-trained `csebuetnlp/banglabert` model. BanglaBERT contains 12 Transformer layers and produces a 768-dimensional contextual representation.

Unlike the BiLSTM pipeline, BanglaBERT does not use the General or Medical FastText embeddings. The input is first normalized and passed through the BanglaBERT tokenizer. Since the tokenizer can divide one original word into multiple subwords, an alignment step is required for NER.

For example, one original word may be represented as

$$w_i \rightarrow (s_{i1}, s_{i2}, \dots, s_{ik}).$$

The NER annotations are defined for w_i , not for every subword. Therefore, the hidden representation of the first subword s_{i1} is selected as the representation of the original word. This produces one 768-dimensional contextual vector for every original token while preserving alignment with the BIO annotations.

Figure 3 compares the two encoder designs.

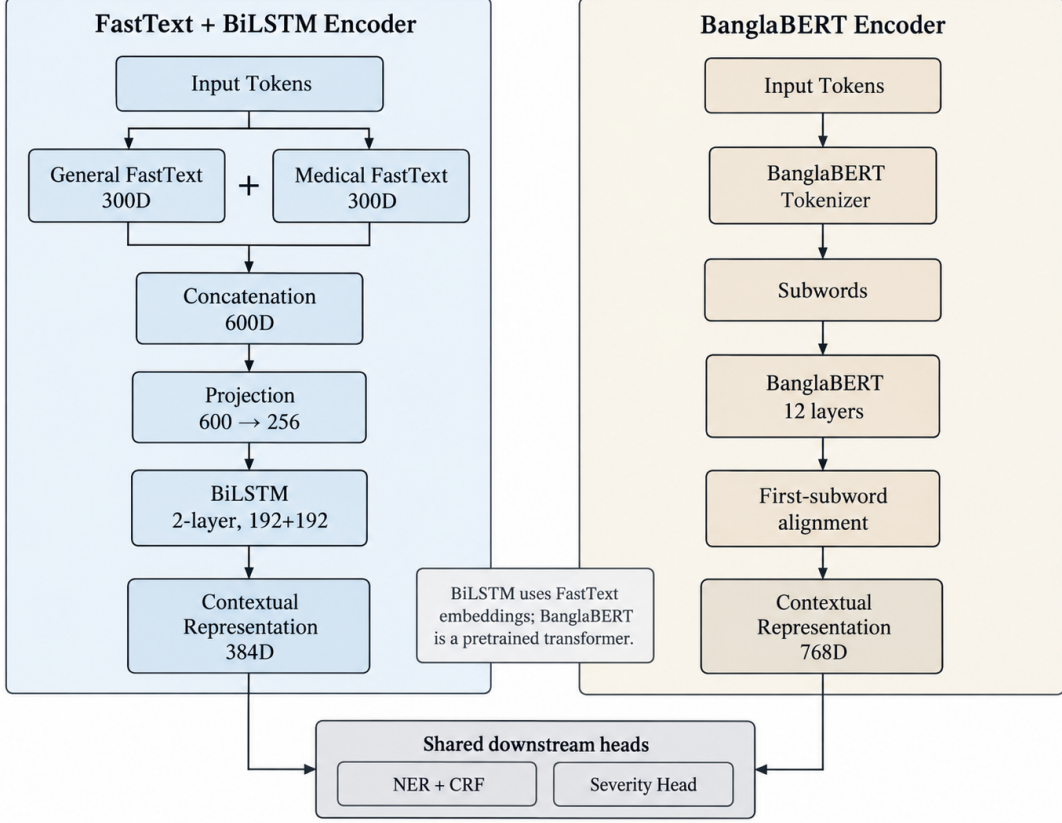


Figure 3: Comparison of the FastText-BiLSTM and BanglaBERT encoders.

4.4 NER and Entity-Aware Severity Prediction

After contextual encoding, the model separates into the NER and severity branches.

For every contextual token representation $\mathbf{h}_i$, the NER linear layer produces 15 scores corresponding to the BIO labels:

$$\mathbf{e}_i = W_{NER}\mathbf{h}_i + b_{NER}. \quad (5)$$

The 15 labels consist of a beginning and inside tag for each of the seven medical entity types together with the outside tag:

$$7 \times 2 + 1 = 15.$$

The linear-layer outputs are called *emission scores*. The CRF combines these emissions with learned transition scores between successive BIO tags. Instead of selecting the highest-scoring label independently for every token, Viterbi decoding finds the highest-scoring complete BIO sequence. The decoded sequence is then converted into the final medical entities.

In parallel, the same emission scores are converted into probabilities using softmax. The beginning and inside probabilities belonging to the same entity type are added together. For example,

$$q_i^{Symptom} = P(B-Symptom | i) + P(I-Symptom | i). \quad (6)$$

Repeating this process for all seven entity categories creates a 7-dimensional vector

$$\mathbf{q}_i \in \mathbb{R}^7,$$

which describes how likely token i is to belong to each medical entity type.

The severity branch therefore receives two forms of information:

- $\mathbf{h}_i$: the contextual meaning of token i ;
- $\mathbf{q}_i$: its medical entity probabilities.

Entity-aware attention combines both representations when calculating the importance of each token:

$$s_i = \mathbf{v}^T \tanh(W_h \mathbf{h}_i + W_q \mathbf{q}_i). \quad (7)$$

The scores are normalized across the sentence:

$$\alpha_i = \frac{\exp(s_i)}{\sum_j \exp(s_j)}. \quad (8)$$

The resulting sentence representation is

$$\mathbf{h}_{att} = \sum_i \alpha_i \mathbf{h}_i. \quad (9)$$

This allows the classifier to give greater attention to medically informative tokens rather than treating all words equally.

A global max-pooled representation is also extracted from the contextual vectors:

$$\mathbf{h}_{max} = \text{MaxPool}(\mathbf{h}_1, \dots, \mathbf{h}_n).$$

The attention and max-pooled representations are concatenated:

$$\mathbf{h}_{final} = [\mathbf{h}_{att}; \mathbf{h}_{max}]. \quad (10)$$

For the BiLSTM model this produces

$$384 + 384 = 768$$

dimensions, while BanglaBERT produces

$$768 + 768 = 1536.$$

The final representation passes through a 256-unit hidden layer, GELU activation, dropout, and a four-unit output layer. The four output logits represent Emergency, Urgent, Routine, and General Query.

Figure 4 summarizes the relationship between the NER, CRF, entity probabilities, and severity branch.

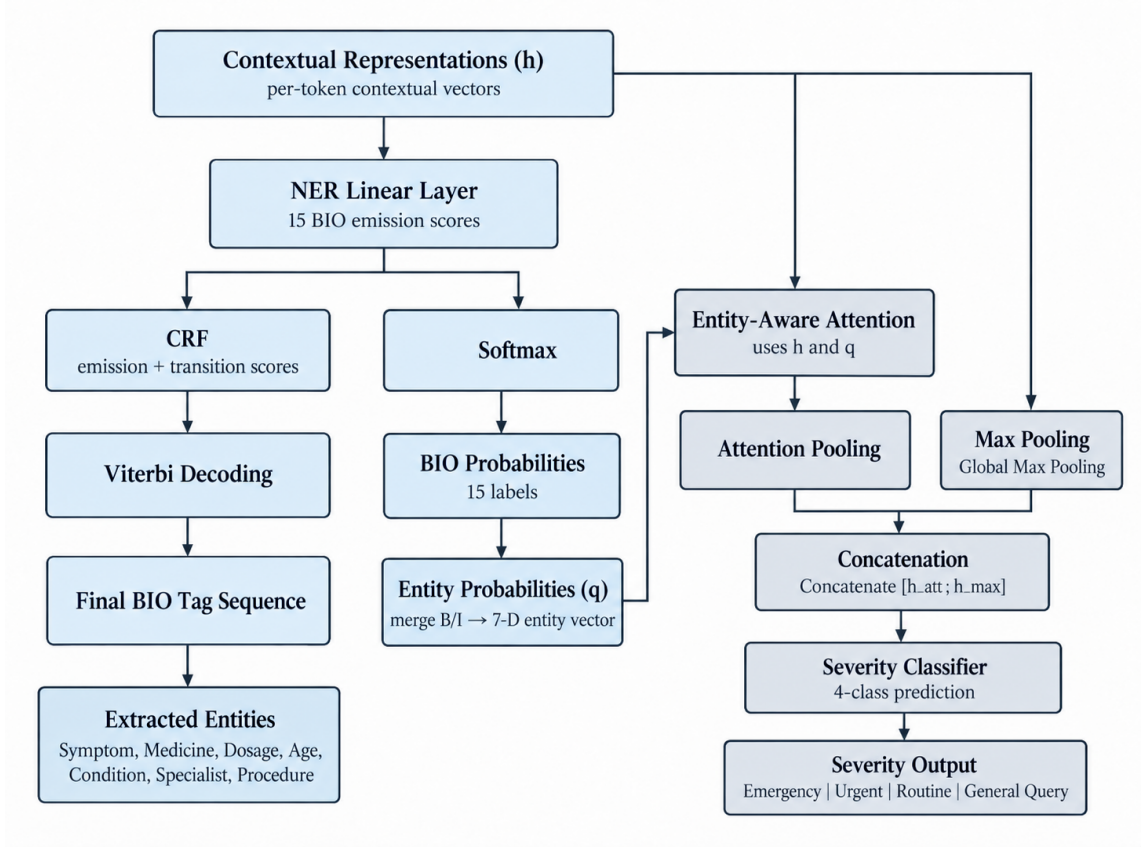


Figure 4: NER, CRF, entity probability extraction, and entity-aware severity classification.

5 Training and Experimental Setup

5.1 Training Procedure

Before training the multi-task architecture, the Medical FastText model is trained separately on the constructed medical corpus. Skip-gram training is used so that a word representation is learned from the surrounding words in which it occurs. FastText additionally represents words using character n-grams, allowing related spellings and rare medical terms to share subword information.

The main architectural and training configurations of the two models are summarized in Table 4.

Table 3: Main architectural and training configuration.

Setting	FastText–BiLSTM	BanglaBERT
Input representation	General FastText 300D + Medical FastText 300D	BanglaBERT subwords
Projection	600 $\rightarrow$ 256	–
Projection activation	GELU	–
Normalization	LayerNorm	–
Projection dropout	0.20	–
Context encoder	2-layer BiLSTM, 192 + 192	12-layer BanglaBERT
Contextual dimension	384	768
NER output	15 BIO scores + CRF	15 BIO scores + CRF
Severity representation	384 + 384 = 768	768 + 768 = 1536
Severity hidden layer	256, GELU	256, GELU
Severity dropout	–	0.40
NER batch size	32	16
NER-stage encoder LR	1×10^{-3}	2×10^{-5}
NER-stage head LR	1×10^{-3}	1×10^{-3}
NER training epochs	up to 20	5
Joint encoder LR	2×10^{-4}	1×10^{-5}
Joint NER-head LR	5×10^{-4}	5×10^{-4}
Joint severity-head LR	1×10^{-3}	1×10^{-3}
Optimizer	AdamW	AdamW
Gradient clipping	1.0	1.0
Weight decay	–	0.01
LR schedule	–	10% warm-up + linear decay

Training of the main neural architecture is performed in three stages:

NER Training $\rightarrow$ Severity-Head Training $\rightarrow$ Joint Fine-Tuning

Stage 1: NER training. The contextual encoder, NER linear layer, and CRF are trained using the Bangla-HealthNER training set. The severity head is not trained during this stage. The NER objective is the CRF negative log-likelihood:

$$\mathcal{L}_{NER} = -\log P(\mathbf{y} \mid \mathbf{x}). \quad (11)$$

This stage first teaches the shared encoder to produce representations useful for identifying medical entities.

Stage 2: Severity-head training. After NER training, the contextual encoder and NER components are temporarily frozen. Only the entity-aware severity head is trained using the severity training set.

The severity objective is four-class cross-entropy:

$$\mathcal{L}_{sev} = -\log P(y_{true} \mid \mathbf{x}). \quad (12)$$

Freezing the encoder at this stage prevents the initially untrained severity head from immediately changing the useful medical representations learned during NER training.

Stage 3: Joint fine-tuning. The shared encoder is then unfrozen. Training alternates between NER batches and severity batches. For an NER batch,

$$\text{NER batch} \rightarrow \mathcal{L}_{NER} \rightarrow \text{encoder} + \text{NER update},$$

whereas for a severity batch,

$$\text{severity batch} \rightarrow \mathcal{L}_{sev} \rightarrow \text{encoder} + \text{severity-head update}.$$

Thus, the shared encoder receives supervision from both tasks, while each task-specific head is updated by its corresponding loss.

The severity branch uses the NER probability vector $\mathbf{q}$, but this connection is detached during backpropagation:

$$\mathbf{q}_{sev} = \text{detach}(\mathbf{q}). \quad (13)$$

This means that severity prediction can use NER information, but severity loss cannot directly modify the NER emission layer through $\mathbf{q}$. The shared encoder can still receive gradients from the severity branch through its contextual representation $\mathbf{h}$.

During joint training, validation NER performance is monitored as a guardrail. A checkpoint is rejected if its validation entity F1 decreases by more than 0.01 from the corresponding pre-joint NER baseline. This prevents improvements in severity classification from being obtained at the cost of substantial degradation in medical entity recognition.

5.2 Experimental Configuration and Calibration

AdamW is used to optimize the neural models. Different learning rates are used for the shared encoder and the task-specific heads. In particular, the pretrained BanglaBERT encoder receives a much smaller learning rate because it already contains pretrained language knowledge.

The main settings are shown in Table 4.

Table 4: Main experimental configuration.

Setting	FastText–BiLSTM	BanglaBERT
Contextual dimension	384	768
BiLSTM layers	2	–
Transformer layers	–	12
NER batch size	32	16
Initial encoder LR	1×10^{-3}	2×10^{-5}
Joint encoder LR	2×10^{-4}	1×10^{-5}
Joint NER-head LR	5×10^{-4}	5×10^{-4}
Joint severity-head LR	1×10^{-3}	1×10^{-3}
Optimizer	AdamW	AdamW
Gradient clipping	1.0	1.0

For BanglaBERT, a 10% warm-up period followed by linear learning-rate decay is used during fine-tuning. Warm-up gradually increases the learning rate at the beginning of training, reducing the risk of unstable updates to the pretrained Transformer.

After the severity model is trained, temperature scaling is applied to its logits. If $\mathbf{z}$ represents the four severity logits, the calibrated probabilities are

$$P(y) = \text{softmax}\left(\frac{\mathbf{z}}{T}\right). \quad (14)$$

The scalar temperature T is learned from validation predictions only. When $T > 1$, the resulting probability distribution becomes less sharp, reducing overconfidence. Temperature scaling does not normally change the predicted class because all logits are divided by the same positive value; its purpose is to make the reported confidence score more reliable.

The highest calibrated class probability is used as the prediction confidence. If this confidence is below the selected validation-based threshold, the query is flagged as a low-confidence case for human review.

6 Results and Evaluation

NER performance is evaluated at the entity level using precision, recall, and F1-score. An extracted entity is counted as correct only when both the entity type and its span boundary match the reference annotation. Severity classification is evaluated using accuracy, class-wise precision, recall, F1-score, Macro-F1, and Emergency recall.

For precision P and recall R ,

$$F_1 = 2 \frac{PR}{P + R}. \quad (15)$$

Macro-F1 calculates the F1-score independently for every class and then takes their unweighted average, preventing larger classes from dominating the reported result.

6.1 NER and Severity Performance

Table 5 compares the final held-out test performance of the two architectures.

Table 5: Final comparison of the two architectures.		
Metric	FastText-BiLSTM	BanglaBERT
NER Entity F1	0.6209	0.6252
NER Macro-F1	0.6669	0.6689
Severity Accuracy	90.68%	91.83%
Severity Macro-F1	0.9099	0.9218
Emergency Recall	95.56%	94.07%
Calibrated Test ECE	0.0250	0.0322
Model Parameters	2.09M	~111M

BanglaBERT provides the stronger overall severity classifier. Severity Macro-F1 increases from 0.9099 to 0.9218, while accuracy increases from 90.68% to 91.83%. The improvement in NER is considerably smaller: entity-level F1 increases only from 0.6209 to 0.6252.

The lightweight FastText-BiLSTM model remains competitive despite containing far fewer parameters. It also achieves a slightly higher Emergency recall of 95.56%, compared with 94.07% for BanglaBERT. This shows that the larger Transformer does not outperform the lightweight model on every individual metric.

Table 6 gives the entity-level F1-score for each medical category.

Table 6: Entity-level F1-score by medical entity category.

Entity	FastText-BiLSTM	BanglaBERT
Symptom	0.4649	0.4677
Medicine	0.8458	0.8604
Dosage	0.5510	0.5169
Age	0.7829	0.7806
Health Condition	0.5048	0.5305
Specialist	0.8852	0.8764
Medical Procedure	0.6336	0.6497

BanglaBERT improves Medicine, Health Condition, Medical Procedure, and Symptom extraction, while the BiLSTM performs slightly better for Dosage, Age, and Specialist. Symptom extraction remains the largest NER difficulty for both architectures.

6.2 Severity Errors, Calibration and Human Review

The BanglaBERT severity confusion matrix is shown in Table 7.

Table 7: BanglaBERT severity confusion matrix on the test set.

True Class	Predicted Class			
	Emergency	Urgent	Routine	General
Emergency	127	8	0	0
Urgent	8	124	15	0
Routine	0	12	114	0
General Query	0	0	0	118

The dominant confusion occurs between Urgent and Routine queries. Among 135 Emergency examples, BanglaBERT correctly identifies 127 and classifies the remaining eight as Urgent. None of the Emergency cases are classified as Routine or General Query. General Query examples are correctly separated in this test set.

For BanglaBERT, temperature scaling learns

$$T = 1.2142.$$

Since $T > 1$, the calibrated softmax probabilities are slightly less confident than the original predictions. Validation Expected Calibration Error (ECE) decreases from 0.0195 before calibration to 0.0119 after calibration, indicating improved agreement between confidence and observed correctness.

The calibrated confidence is also used for human-review routing. With the selected confidence threshold of approximately 0.9392, 143 of the 526 BanglaBERT test queries are marked as low-confidence cases. These flagged queries contain 39 of the 43 classification errors, meaning that approximately

$$\frac{39}{43} \times 100 \approx 90.7\%$$

of the observed severity errors are captured by the review mechanism.

The confidence threshold therefore does not change the predicted severity class; instead, it provides an additional indication of which predictions are less reliable and may benefit from manual review.

7 Error Analysis and Discussion

The quantitative results show that severity classification is considerably stronger than exact-span medical NER. Error analysis was therefore performed to determine where the remaining failures occur and to understand the differences between the two encoder architectures.

A major NER difficulty is **Symptom** extraction. Although symptoms are frequent in the dataset, their spans are often long and linguistically variable. Many errors are boundary errors rather than complete failures to recognize the medical concept. For example,

Reference Entity	Possible Prediction
<i>severe chest pain</i>	<i>chest pain</i>

Both predictions identify the main symptom, but exact entity-level evaluation counts the second span as incorrect because its boundary does not fully match the reference annotation. This contributes to the relatively low Symptom F1 of 0.4649 for FastText–BiLSTM and 0.4677 for BanglaBERT.

The severity confusion matrix shows a different pattern. The largest classification confusion occurs between **Urgent** and **Routine**, which are neighbouring severity categories and can contain similar symptoms. In contrast, BanglaBERT separates General Query examples cleanly in the test set. Emergency recognition is also strong: 127 of 135 Emergency examples are correctly classified by BanglaBERT, while the remaining eight are classified as Urgent rather than the lower-severity Routine or General Query classes.

Negation is another important source of difficulty. For example,

“I do not have chest pain or shortness of breath.”

contains medically important expressions such as *chest pain* and *shortness of breath*, which the NER component may correctly recognize as symptoms. However, the words indicating absence or negation must also be understood by the severity classifier. The current architecture does not explicitly model whether an extracted symptom is present, absent, or uncertain, which can lead to incorrect severity estimates.

Mixed Bangla–English and Banglish queries introduce additional variation. FastText partially addresses this problem through character n-grams, while BanglaBERT provides stronger contextual representations. Nevertheless, highly informal transliterations and uncommon spelling forms remain more difficult than standard Bengali text.

The comparison between the two encoders reveals an important trade-off. BanglaBERT improves severity Macro-F1 from 0.9099 to 0.9218, but the NER entity-level F1 increases only from 0.6209 to 0.6252. Therefore, the Transformer provides a clearer improvement for query-level severity classification than for exact medical entity extraction. This suggests that stronger contextual representations are particularly useful for understanding the overall meaning of a health query, while exact entity boundaries remain dependent on the quality and consistency of token-level annotations.

Model size must also be considered. The FastText–BiLSTM model contains approximately 2.09 million trainable parameters, while BanglaBERT contains approximately 111 million. Despite this large difference, the lightweight model remains competitive and achieves slightly higher Emergency recall (95.56% versus 94.07%). Therefore, the appropriate model depends on whether the priority is maximum overall severity performance or lower computational cost.

The confidence mechanism provides an additional safety-oriented output. BanglaBERT flags 143 of the 526 test queries as low-confidence cases using the selected threshold. These cases contain 39 of the 43 observed severity errors. Thus, confidence calibration does not improve the class prediction itself, but it provides useful information about which predictions are more uncertain.

Figure 5 shows an example of the application output, including extracted entities, class probabilities, calibrated confidence, and the human-review recommendation.

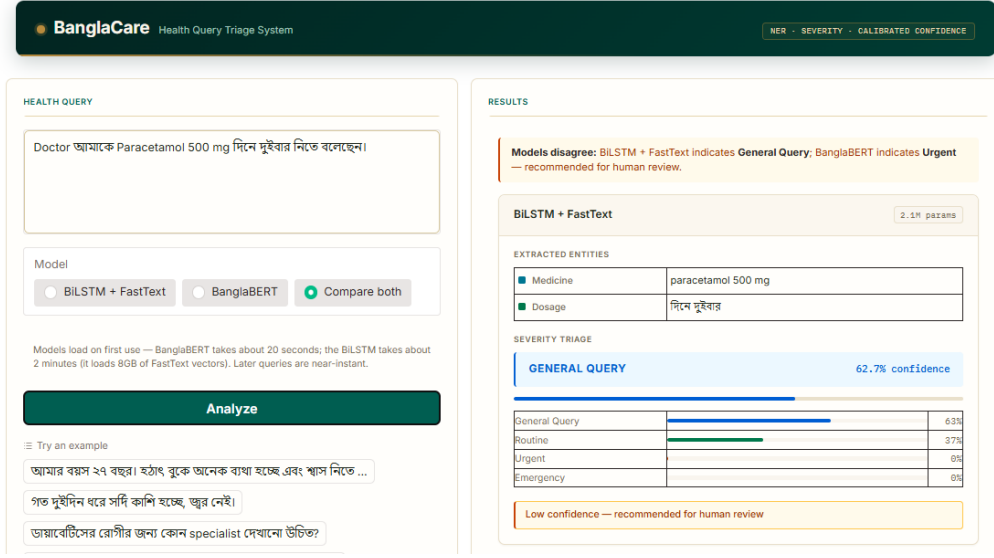


Figure 5: Example BanglaCare prediction showing extracted entities, severity probabilities, calibrated confidence, and review status.

8 Limitations and Future Work

Although BanglaCare achieves strong severity classification performance, several limitations remain. First, exact-span NER performance, particularly for symptoms, is substantially weaker than severity classification. Future work can investigate improved sequence-tagging methods, span-based NER, or additional annotation cleaning to reduce boundary errors.

Second, negation is not explicitly represented. The current NER branch can identify a symptom such as *chest pain*, but does not separately encode whether that symptom is present or negated.

Third, more naturally occurring Banglish and mixed-language consumer queries would improve evaluation of real-world robustness. Additional spelling variation and transliteration data could also improve handling of informal health queries.

The current comparison is primarily based on one fixed experimental split and training seed. Repeating training with multiple random seeds and reporting mean and standard deviation would provide stronger evidence that observed differences between FastText-BiLSTM and BanglaBERT are consistent.

An additional ablation is also important for validating the entity-aware design. A future experiment should compare the same severity classifier *with* and *without* the entity probability vector $\mathbf{q}$. This would directly measure how much medical NER information contributes to severity prediction.

Finally, the current system should be considered a decision-support prototype rather than an autonomous medical diagnostic system. Further evaluation using larger real-world datasets and expert-reviewed clinical scenarios would be required before practical healthcare deployment.

9 Conclusion

This project developed BanglaCare, a multi-task NLP system for medical entity recognition and severity triage of Bengali consumer health queries. The system identifies seven medical entity categories and classifies each query as Emergency, Urgent, Routine, or General Query.

Two contextual encoders were evaluated. The lightweight architecture combines General and Medical FastText embeddings with a two-layer BiLSTM, while the second architecture uses pretrained BanglaBERT. Both models share a CRF-based NER branch and an entity-aware severity classifier in which soft NER probabilities contribute to attention-based severity prediction.

On the held-out test data, FastText–BiLSTM achieved an NER entity F1 of 0.6209 and severity Macro-F1 of 0.9099, while BanglaBERT achieved 0.6252 and 0.9218 respectively. BanglaBERT therefore provided the stronger overall severity performance, although the much smaller BiLSTM model remained competitive and achieved slightly higher Emergency recall.

The results also show that severity classification is currently more reliable than exact-span entity extraction, with symptom boundaries representing the main NER difficulty. Confidence calibration and review routing further allow uncertain predictions to be identified without changing the predicted severity class.

Overall, the project demonstrates that medical entity recognition and severity classification can be integrated within a common architecture, and that entity information can be explicitly incorporated into query-level triage rather than treating the two NLP tasks independently.

References

- [1] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, “Enriching Word Vectors with Subword Information,” *Transactions of the Association for Computational Linguistics*, vol. 5, pp. 135–146, 2017.
- [2] A. Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, 2017.